Start Docker when you're ready.

While everyone gets settled:

Ran preflight? You’re ready.
Start Docker, then open a terminal in the repo folder and a browser.

Couldn’t download?
Raise a hand for a USB bundle; skip the conference Wi-Fi.

Laptop can’t run the model?
Use the backup-cloud QR when a proctor directs you.

Keep the walkthrough open
workshop-steps.md explains every step, and why.

github.com/bbenz/
stack-it-yourself-workshop

Repo · walkthrough · cheatsheet

AGNTCON + MCPCON NORTH AMERICA 2026 · HANDS-ON WORKSHOP

Stack It Yourself

Open infrastructure for AI agents, from Compose to cluster

Flexible-format hands-on workshop · follow the checkpoints at your room's pace

Brian Benz Principal AI Advocate, Microsoft

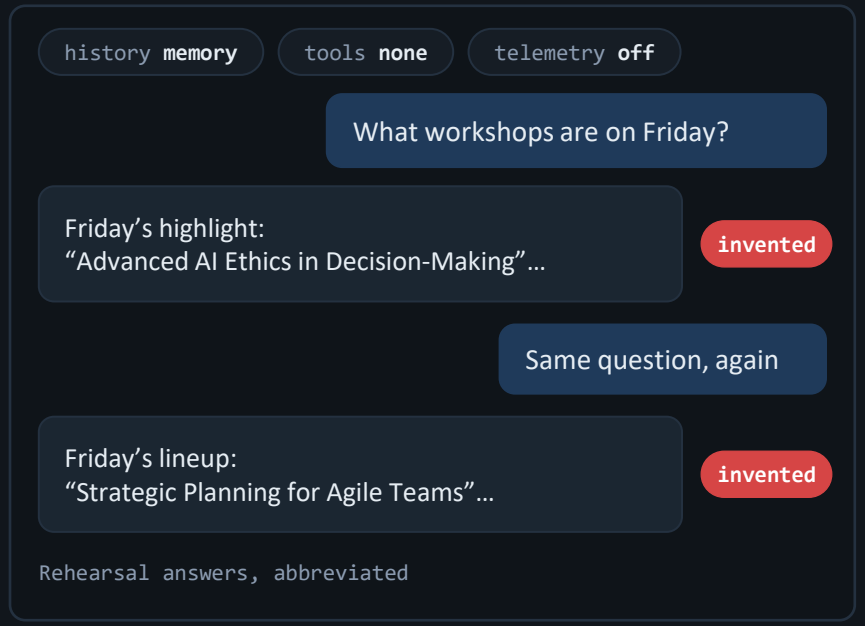
Pamela Fox Principal Cloud Advocate, Microsoft

6	**Kubernetes (homework)**
5	MCP tools
4	Redis history
3	OpenTelemetry + LGTM
2	Responses API
1	Agent on local inference

One layer per step.
The agent code never changes.

STEP 0 · COLD OPEN

Fluent is not the same as right



NOW BREAK IT THREE WAYS

1. **Take away inference**
   `docker compose stop ollama`
   → a red error: no model, no answer

2. **Ask why it took 9 seconds**
   `telemetry off`
   → there is no data to look at

3. **Restart the agent**
   `docker compose restart agent`
   → the conversation is gone

Each break is an infrastructure decision, not a model decision.

THE THESIS

Your day-one infrastructure choices decide whether day 100 is an extension or a rewrite.

Every one of those choices can be made with open components that run the same way on a laptop and in a cluster.

No GPU

No cloud account

No proprietary API key

Every component is open source: MIT, Apache-2.0 or AGPL-3.0.

THE FOUR DECISIONS

Four infrastructure decisions



Steps 1–2

Where does inference come from?

WE BUILD

An OpenAI-compatible server you run: Ollama

WE AVOID

A vendor SDK and a vendor API key



Step 3

How do you see what the agent did?

WE BUILD

OTel GenAI conventions + Grafana LGTM

WE AVOID

A vendor dashboard and a vendor schema



Step 4

Where does the conversation live?

WE BUILD

Redis, behind a serialization boundary

WE AVOID

Process memory or provider-side threads



Step 6

How does it reach production?

WE BUILD

The same image, from Compose to Kubernetes

WE AVOID

A rewrite for a hosted runtime

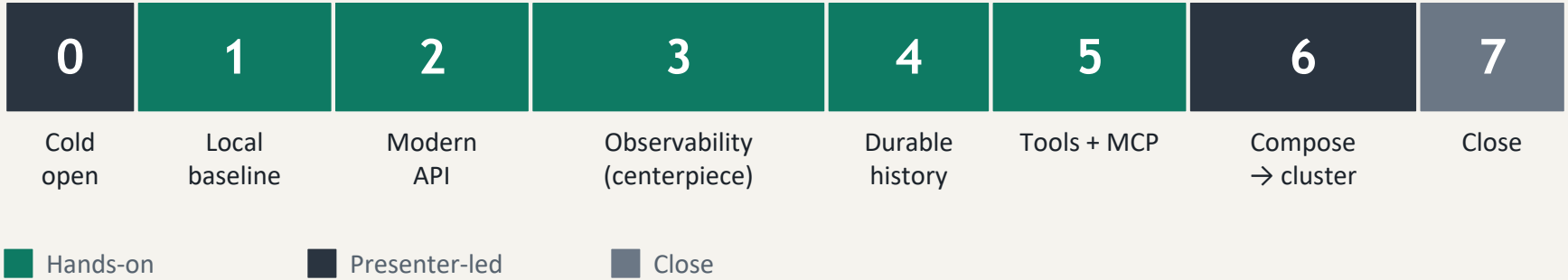


Out of scope: governance, threat modeling and evals — and no claim that one Compose file is production-grade.

WORKSHOP PATH

Seven layers, one portable agent

Move at the room's pace; every checkpoint is a safe pause.



5

hands-on steps,
each with a checkpoint

1 + 1

file to copy and command to run,
per step

0

lines of agent code changed,
laptop to cluster

HOW EVERY STEP WORKS

The code never changes

Only **.env** does. Every step is the same two moves:

```
cp env/stepN.env .env
docker compose up -d
```

WHERE N IS

`1 baseline` `2 Responses` `3 telemetry` `4 Redis` `5 tools`

Each step changes two settings or fewer. The agent's code is identical in every step — and in the cluster.

See what a step turns on

```
git diff --no-index env/step2.env env/step3.env
```

Fallen behind?

Copy the room's step file and run `up -d`.

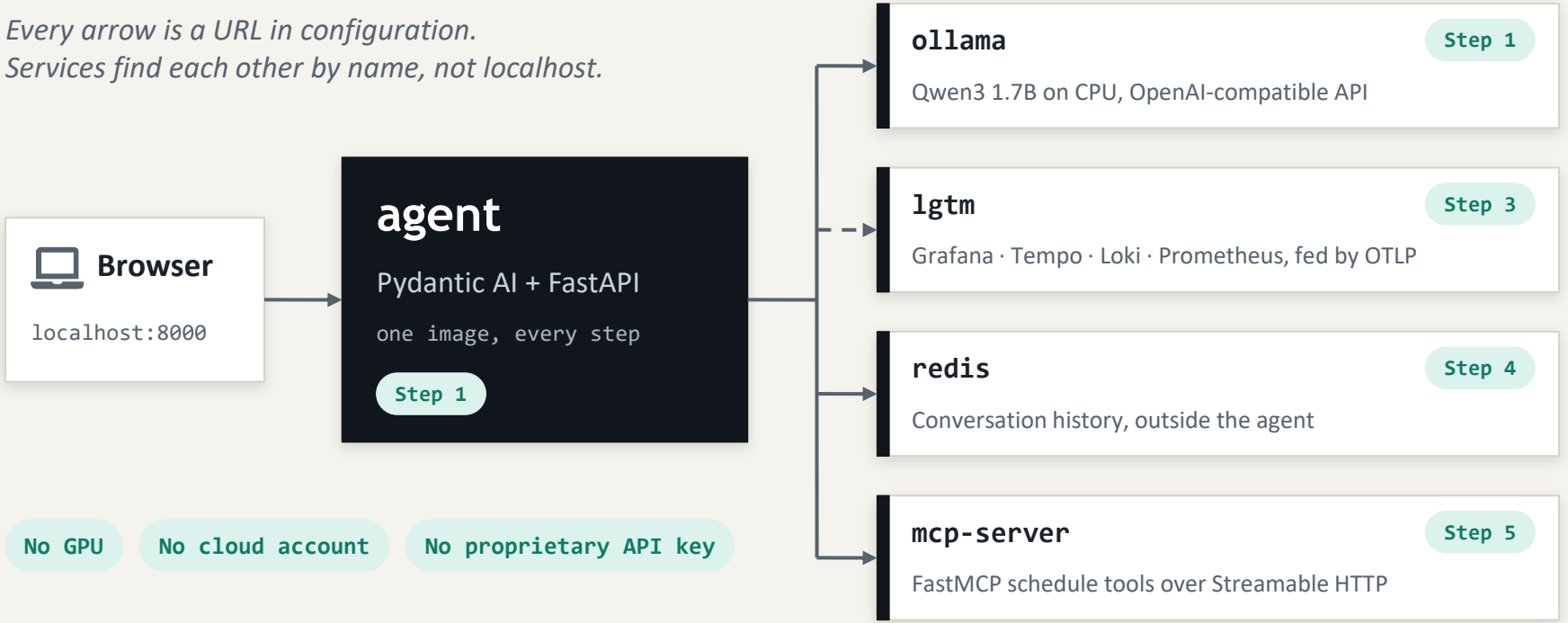
On Windows?

`cp` works in PowerShell. In cmd, use `copy`.

ARCHITECTURE

What you'll build

Every arrow is a URL in configuration.
Services find each other by name, not localhost.



No GPU · No cloud account · No proprietary API key

STEP 1 · BASELINE

An agent on open inference

API **Chat Completions** | history **memory** | tools **none** | telemetry **off**

RUN

```
cp env/step1.env .env
docker compose up -d
```

LOOK FOR

model **ollama:qwen3:1.7b** | class **OllamaModel**

API **Chat Completions** | telemetry **off**

THEN

Open **http://localhost:8000**

Ask anything, e.g. *“What is a container?”*

First reply: 15–30 s
The model is loading into memory. Press Send once and wait; after that, replies are much faster.

CHECKPOINT 1 Reply in the browser · `docker compose ps`: ollama + agent healthy · hard gate

STEP 1 · THE SEAM

The seam is a string

API **Chat Completions** | history **memory** | tools **none** | telemetry **off**

```
MODEL=ollama:qwen3:1.7b
```

provider prefix
picks class + API

model name
what the server should run

MODEL prefix	Pydantic AI class	API surface
ollama:	OllamaModel	Chat Completions
openai-chat:	OpenAIChatModel	Chat Completions
openai-responses:	OpenAIResponsesModel	Responses

REASONING=off

sends openai_reasoning_effort=none

10.6 s → 2.8 s

for one tool-using question

243 → 34

output tokens

qwen3:1.7b with tools on, 16-thread laptop CPU

agent/app/assistant.py → Agent(model, instructions, tools, toolsets, model_settings)

STEP 2 · MODERN API

Same server, modern API

API **Responses** | history **memory** | tools **none** | telemetry **off**

THE WHOLE DIFF

```
-MODEL=ollama:qwen3:1.7b
+MODEL=openai-responses:qwen3:1.7b
```

REFRESH, THEN LOOK FOR

class **OpenAIResponsesModel** | API **Responses**

RUN

```
cp env/step2.env .env
docker compose up -d
```

IN THE AGENT LOG

```
POST http://ollama:11434/v1/responses
"HTTP/1.1 200 OK"
```

Two API surfaces, one model: same weights, same Ollama, same agent code.

CHECKPOINT 2 The chips show OpenAIResponsesModel and Responses, and a reply arrives.

STEP 2 · BACKUP CLOUD OPTION

“OpenAI-compatible” is a spectrum

Laptop
`Ollama · qwen3:1.7b`

↓

Microsoft Foundry
qwen3-4b-instruct · hosted backup

SAME DEPLOYMENT · SECONDS PER TURN



- This endpoint speaks both APIs: no 400.
- Its per-deployment route takes only an `api-key` header; the SDK’s Bearer auth gets a 401.
- Some catalogs answer the Responses API with a flat 400.

The base URL gets you most of the way.
The details are why you test — and why you trace.

STEP 3 · OBSERVABILITY

Turn the lights on

API **Responses** | history **memory** | tools **none** | telemetry **on**

THE DIFF

```
-COMPOSE_PROFILES=
+COMPOSE_PROFILES=observability
-OTEL_SDK_DISABLED=true
+OTEL_SDK_DISABLED=false
```

RUN

```
cp env/step3.env .env
docker compose up -d
```

Grafana → **http://localhost:3000** (no login)

ONE CONTAINER: grafana/otel-lgtm

Grafana dashboards

Tempo traces

Loki logs

Prometheus metrics

plus an OpenTelemetry collector receiving OTLP on :4318

IN THE CODE SINCE STEP 1

```
Agent.instrument_all(InstrumentationSettings(
    include_content=settings.capture_content))
```

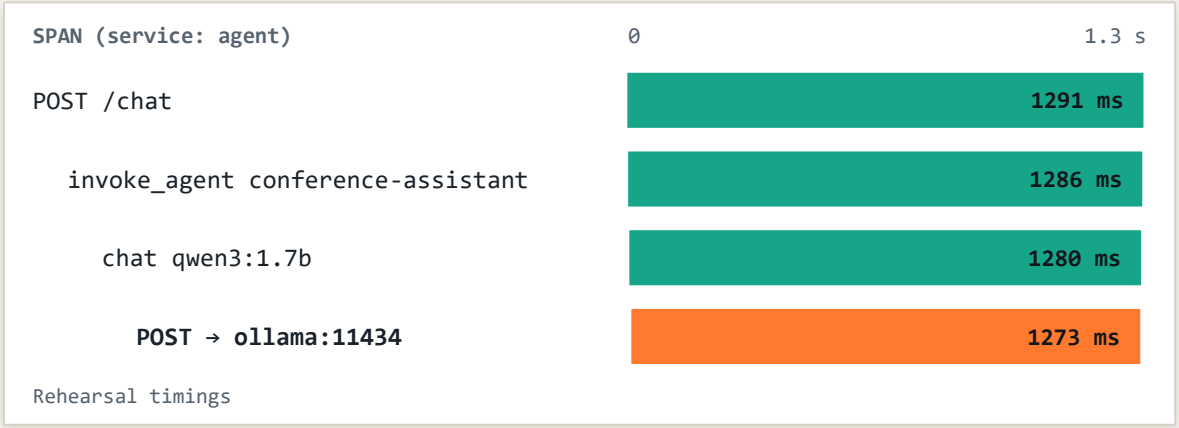
CAPTURE_CONTENT=true puts prompts, completions and tool data in spans. Dev only.

STEP 3 · READ THE TRACE

Read the agent loop

API **Responses** | history **memory** | tools **none** | telemetry **on**

Ask a question → click **trace ↗** under the reply → Grafana opens on that exact trace



The agent loop
request → agent → model → HTTP

≈99%: the model call
latency lives in inference

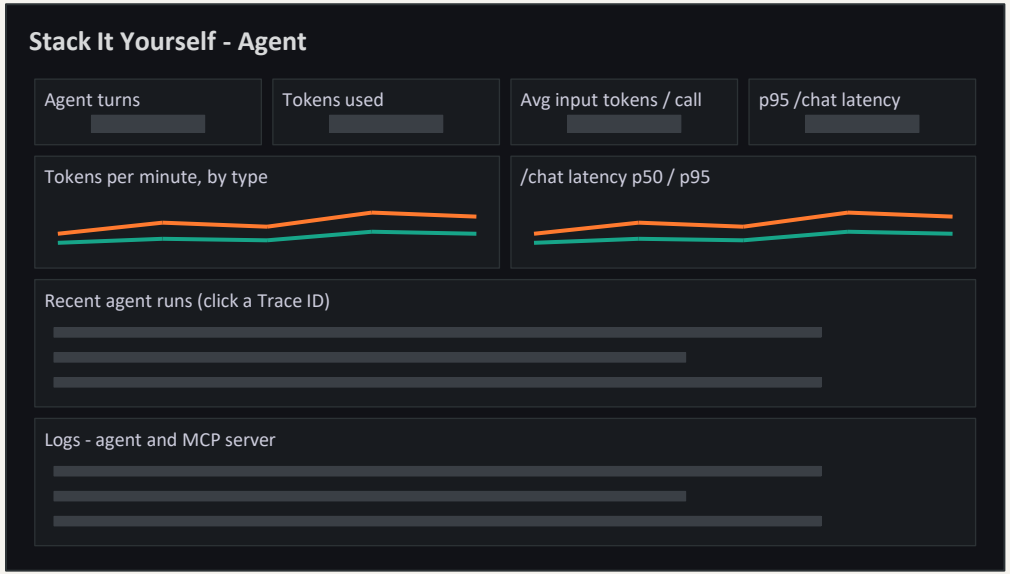
gen_ai.* attributes
OpenTelemetry GenAI conventions: model, tokens, messages

CHECKPOINT 3 Point at a span in your own trace and say what the agent was doing.

STEP 3 · METRICS AND LOGS

Three signals, one trace ID

`API` **`Responses`** `history` **`memory`** `tools` **`none`** `telemetry` **`on`**



Schematic of the provisioned dashboard: eight panels

84 → 124 → 147

input tokens per call, turn by turn

Every turn resends the whole history.
History costs tokens, and now you can see it.

Logs carry trace_id

Expand a log line and follow its link to the trace.

Search lags; links don’t

Tempo’s search index needs about 30 s. The trace ↗ link opens by ID right away.

STEP 4 · DURABLE HISTORY

Kill the container, keep the chat

API **Responses** | history **redis** | tools **none** | telemetry **on**

THE DIFF

```
-COMPOSE_PROFILES=observability
+COMPOSE_PROFILES=observability,state
-HISTORY_BACKEND=memory
+HISTORY_BACKEND=redis
```

RUN

```
cp env/step4.env .env
docker compose up -d
```

THE DEMO

1. Chat: “My name is Pat, and I’m in room LL21 DEF.”
2. `docker compose rm -sf agent` → memory gone
3. `docker compose up -d` → a brand-new container
4. Reload: the chat is back; “served by” has changed
5. Ask: “What’s my name, and which room am I in?”

CHECKPOINT 4 Earlier messages are back: same conversation ID, new “served by” container.

STEP 4 · THE NUMBERS

State costs 2 ms. Inference costs 700.

load history
LRANGE

0.6 ms

save this turn
RPUSH · LTRIM · EXPIRE

1.3 ms

model call
chat qwen3:1.7b

721 ms

```
POST /chat                  729 ms
├─ LRANGE                   0.6 ms   ← load history
├─ invoke_agent             724 ms
│  └─ chat qwen3:1.7b       721 ms
└─ RPUSH LTRIM EXPIRE       1.3 ms   ← save turn
```

Every turn of a conversation, across restarts:

```
{ span.gen_ai.conversation.id = "<id>" }
```

TraceQL in Grafana Explore → Tempo

Moving history out of the process is nearly free — and it's what makes Step 6 easy.

STEP 5 · TOOLS + MCP

Give the agent real facts

API **Responses** | history **redis** | tools **local, mcp** | telemetry **on**

THE DIFF

```
-COMPOSE_PROFILES=observability,state
+COMPOSE_PROFILES=observability,state,tools
-TOOLS=
+TOOLS=local,mcp
```

RUN

```
cp env/step5.env .env
docker compose up -d
```

THREE TOOLS, TWO PLACES

`current_time(city)`	inside the agent process
`list_workshops(day)`	in mcp-server, over MCP
`search_workshops(keyword)`	in mcp-server, over MCP

Ask again: *"What workshops are on Friday?"*

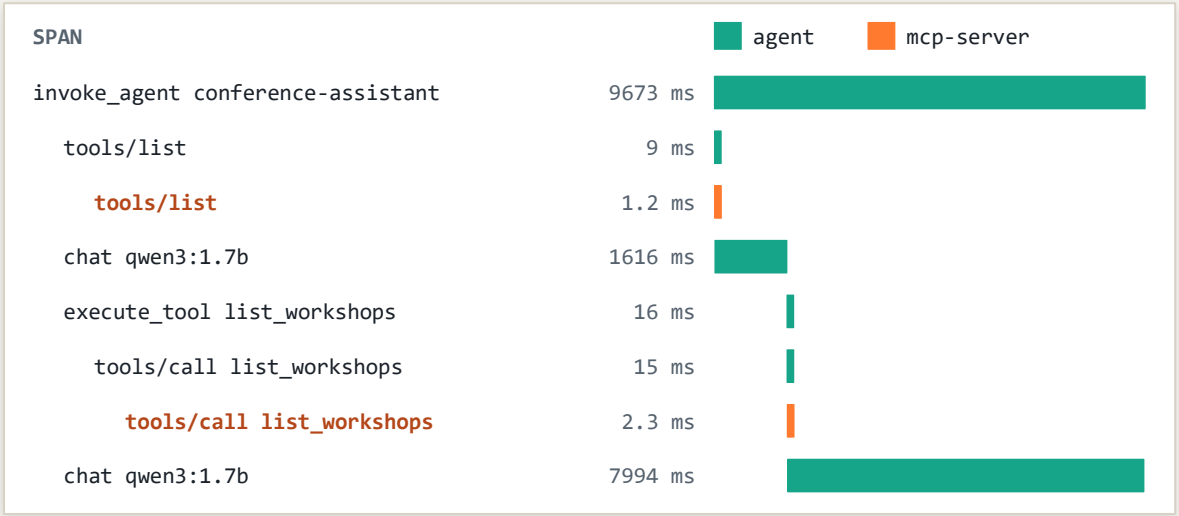
```
tool list_workshops()
10:25 Stack It Yourself · 12:10 Governing AI Agent Actions
16:00 Secure Agentic Framework (SAF)
```

Same question as the cold open. Then it guessed; now it knows.

STEP 5 · THE CROSS-PROCESS TRACE

One trace, two processes

API **Responses** | history **redis** | tools **local, mcp** | telemetry **on**



15 ms

MCP round trip
(2.3 ms in the server)

8 s

writing the answer

SEP-414
The W3C traceparent rides in the MCP request's _meta field.

CHECKPOINT 5 A trace that shows both agent and mcp-server.

STEP 5 · A FINDING WORTH SHARING

Prompt wording is infrastructure

API **Responses** | history **redis** | tools **local, mcp** | telemetry **on**

FIRST WORDING

2 / 4

real tool calls

"…call a tool instead of guessing."

The model wrote "Call the search_workshops function…" as text instead of calling it.

REWRITTEN

8 / 8

real tool calls in the full stack

"…use a tool to get the facts first… Never describe a tool call in your reply - just make it."

Tool wording appears only when tools are on.

qwen3:1.7b with thinking off. The traces are how you'd find a problem like this.

STEP 6 · COMPOSE → CLUSTER · PRESENTER DEMO

Compose maps to Kubernetes

`API Responses` `history redis` `tools local, mcp` `telemetry on` `served by agent-9fd9f4897-9rxjg`

Compose	Kubernetes (k8s/)	Why it matters
`agent service`	**Deployment + Service**	scales because history lives in Redis
`depends_on + healthcheck`	**readiness and liveness probes**	probes gate traffic; no startup ordering
`.env + environment`	**ConfigMap + Secret**	base URLs become cluster DNS names
`./ollama-data`	**PersistentVolumeClaim**	model weights are the big one
`published ports`	**NodePort (Ingress in production)**	agent :8081 · Grafana :3001
`otel-lgtm container`	**the same all-in-one image here**	production splits it: a collector plus Tempo, Loki, Prometheus and Grafana

Same image, no rebuild: `cluster-up` loads it from your Docker cache into the `kind` cluster.

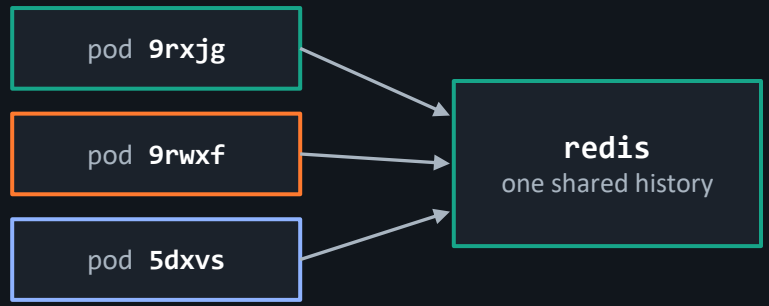
STEP 6 · SCALE IT

Three pods, one conversation

```
kubectl -n stack-it-yourself scale deployment/agent --replicas=3
```

ONE CONVERSATION, SIX TURNS (REHEARSAL)

turn 1	agent-…-9rxjg	Hello, Pat! …
turn 2	agent-…-9rwxf	*(reply not shown)*
turn 3	agent-…-9rxjg	Your name is Pat.
turn 4	agent-…-9rwxf	You like observability.
turn 5	agent-…-9rxjg	Pat likes observability.
turn 6	agent-…-5dxvs	You're welcome! …



3 distinct pods served one conversation

Same image as Compose: identical config digest

Stateless pods plus shared Redis: scaling is one number in one command.

STEP 6 · BE HONEST

What changes in production

CHANGES

The collector becomes real infrastructure

Batching, tail sampling and redaction;
Tempo, Loki, Prometheus and Grafana run separately.

Redis gets real storage

A StatefulSet or a managed service.
This demo’s cluster Redis has no persistent volume.

Inference moves to a serving tier

vLLM with continuous batching on GPU nodes:
one user turn can be 5–15 model calls.

Ingress, TLS, auth, secrets and limits

None of which this lab sets up.

DOESN'T CHANGE

The agent code.

Only the base URL, the OTLP endpoint and REDIS_URL point somewhere new.

STEP 7 · CLOSE

What we deliberately skipped

Routing, failover, MCP federation, edge policy	→	Keep Infrastructure Out of Your AI Agents and MCP Servers	DEEP DIVE
Capability boundaries, prompt-injection blast radius	→	Break the Lethal Trifecta	DEEP DIVE
Policy, registries, audit, token spend	→	Governing AI Agent Actions: MCP and Beyond	COMPANION
Hardening agentic runtimes on Kubernetes	→	Secure Agentic Framework (SAF)	DEEP DIVE
Evals and regression testing	→	Nobody covers it: your traces are the dataset	next problem

→ **Explore the companion sessions for deeper coverage; your token metrics are raw material for governance.**

TAKE IT FURTHER

Extend it yourself

Bigger model

`MODEL=openai-responses:qwen3:4b-instruct`

Content capture off

`CAPTURE_CONTENT=false`
then see what leaves the spans

Another OTLP backend

Jaeger or your vendor: same spans, no code change.

Redis → Valkey

Use a fresh volume: wire-compatible isn't file-compatible.

A second agent

Trace the handoff between them.

Write an eval

Check a real tool was called, not just a plausible answer.

Details for each: **`workshop-steps.md`** → Where to look next

WRAP-UP

Tear down, take it home

STOP THE STACK

```
docker compose --profile observability --profile state --profile tools down
```

Images and the model stay on disk. Add `--volumes` only if you also want to erase the Redis history.

TAKE IT HOME

- **workshop-steps.md** the full walkthrough, with the why
- **CHEATSHEET.md** every command on one page
- **prerequisites.md** set up any laptop later
- **github.com/bbenz/stack-it-yourself-workshop**



Thank you! Brian Benz · Pamela Fox